

Timothy Delmoro · Justin Basilides · Aian Cuento · Calvin Bellen

DLSUD EventHub — Group Laboratory Examination Submission

GitHub repo <https://github.com/eximno/dlsud-eventhub>

GitHub Pages <https://eximno.github.io/dlsud-eventhub/>

DLSUD EventHub

Group Laboratory Examination Submission

Course: 4th-year BSIT group laboratory examination

Group members: Timothy Delmoro, Justin Basilides, Aian Cuento, Calvin Bellen

Repository: <https://github.com/eximno/dlsud-eventhub>

Live site: GitHub Pages → Settings → Pages → *Deploy from a branch*, `main`, / (root)

How to read the prompts in this document. The repository contains the Task 1 prompt (recorded below, verbatim) but **no separate master-prompt file**, and no chat transcripts. The Task 2, 3 and 4 prompts are therefore *task-specific prompts reconstructed for examination documentation* from the Task 1 prompt's requirements and constraints and from the examination brief. They are not transcripts of five independent AI conversations. Every "AI-Generated Output" section describes the **artefact that is actually in this repository**, and every correction cited in the Verification Log is traceable to a commit (`9f4c5fc`) or to a test/lint run named in §6.5.

Contents

1. Team Roster
 2. Task 1 — Requirements Analysis & Prompt Architecture
 3. Task 2 — AI-Assisted Frontend Development
 4. Task 3 — Database Design & ERD Generation
 5. Task 4 — Shift-Left Testing, Security & Refactoring
 6. Task 5 — Group Integration & Verification Report
-

1. Team Roster

#	Member	Role	Primary Responsibility
Member 1	Timothy Delmoro	Systems Architect & Prompt Lead	Task 1 + Task 5
Member 2	Justin Basilides	Frontend Engineer	Task 2
Member 3	Aian Cuento	Database & Backend Engineer	Task 3
Member 4	Calvin Bellen	QA & Security Engineer	Task 4

2. Task 1 — Requirements Analysis & Prompt Architecture

Owner: Timothy Delmoro

2.1 Objective

Turn the examination brief into a defensible architecture for a 3-hour prototype: a campus event registration system for De La Salle University–Dasmariñas that must be publicly reachable on GitHub Pages (static hosting, no server process, no server-side database).

2.2 Prompt Used

This is the Task 1 prompt as recorded by the team. It uses the RCTC structure — **Role**, **Context**, **Task**, **Constraints** — and carries two explicit negative constraints (React/Express/PostgreSQL/etc. and "no invented features").

ROLE

You are a senior software architect who has shipped small internal web systems for Philippine universities, and who also teaches a 4th-year BSIT laboratory class. You optimise for what a group of students can actually finish, demonstrate and defend.

CONTEXT

Four 4th-year BSIT students have a 3-hour closed laboratory examination. They must deliver a working campus event registration prototype for De La Salle University-Dasmariñas, called DLSUD EventHub, plus a database schema, an ERD, unit tests, and a secure refactor of a deliberately flawed C# data-access method. The finished prototype must be publicly reachable through GitHub Pages, which is static hosting: it cannot run a server process or a server-side database. The examiners will open the site, read the SQL, read the tests, and ask why each decision was made. The students have no budget and no cloud accounts.

TASK

Propose the simplest architecture that satisfies all of the following, and justify every component in one sentence each:

1. students can browse, search and filter campus events;
2. students can open an event and register with a DLSU-D student e-mail;
3. remaining seats are shown and capacity is enforced;
4. duplicate registration is prevented;
5. an admin view lists events, registration counts and attendees;
6. a normalised relational schema (at least 3NF) with keys, constraints and indexes, expressible as a Mermaid ERD;
7. business rules that can be unit tested without a running database;
8. a C# service demonstrating parameterised SQL and correct resource disposal, delivered as a separate exercise rather than as the site's backend.

State clearly which parts run on GitHub Pages and which do not.

CONSTRAINTS

- Prefer vanilla HTML5, vanilla CSS and vanilla JavaScript.
- Use SQLite as the database, executed in the browser through sql.js / WebAssembly for the public prototype.
- Use C# with xUnit for the backend and testing deliverables.
- The whole frontend must run as a static site with relative paths only.
- Accessibility (WCAG 2.1 AA) and responsive layout are requirements, not extras.
- Every component must be defensible in a 5-minute oral explanation.
- **NEGATIVE CONSTRAINT:** do NOT introduce React, Next.js, Vue, Laravel, Express, any Node backend server, MySQL, PostgreSQL, Firebase, Supabase, Redux or any other state-management library, any authentication provider, any CSS framework, any bundler, or any build step. Do not propose containers, CI pipelines or cloud infrastructure. If a requirement seems to need one of these, say so explicitly and propose the simplest alternative instead.
- **NEGATIVE CONSTRAINT:** do not invent features that were not asked for.

2.3 AI-Generated Output

Prompt requested: the simplest architecture, with one-sentence justification per component, and a clear statement of what runs on GitHub Pages.

AI output accepted by the team (this is the architecture the repository implements):

Component	Decision	One-sentence justification
Presentation	Three static HTML pages + one CSS file	Semantic HTML5 and custom properties cover every layout and state requirement with no framework and no build step.
Behaviour	Five small vanilla JS modules	Separating pure rules (<code>validation.js</code> , <code>registration.js</code>) from data access (<code>database.js</code>) and from the DOM (<code>events.js</code> , <code>app.js</code>) is what makes the rules unit testable.
Database	SQLite compiled to WebAssembly via <code>sql.js</code>	Real SQL with real <code>CHECK</code> , <code>UNIQUE</code> and foreign-key constraints, running inside static hosting.
Persistence	The SQLite file exported to <code>localStorage</code>	A registration survives a reload without any server.
Schema source of truth	<code>database/schema.sql</code> + <code>database/seed.sql</code> , fetched at runtime	The SQL the examiners read is literally the SQL the site executes — the two cannot diverge.
Seat maths	A SQL view, <code>event_availability</code>	One definition of "seats left", shared by the catalog, the detail page and the admin dashboard.
Backend exercise	<code>backend/RegistrationService.cs</code> , standalone	GitHub Pages cannot execute C#, so the security deliverable is kept separate and honest rather than pretended into the site.
Tests	xUnit + Moq for C#; <code>node:test</code> for JavaScript	The repository depends on nothing at runtime, and both implementations of the same rules are pinned by tests.
Hosting	GitHub Pages from the repository root	Relative paths only; <code>../database/*.sql</code> resolves when the root is the site root.

What does not run on GitHub Pages: `backend/RegistrationService.cs` and its xUnit tests. They are a separate examination exercise (see Task 4), and the site never calls them.

Team-recorded first attempt. The team recorded that the assistant's first response, before the negative constraints were tightened, proposed a React SPA with an Express/ASP.NET API, PostgreSQL, JWT, Docker Compose and CI, and that it was rejected. *Recorded from the team's memory of the session; the repository holds no transcript that can confirm it (see Verification Log entry 8).*

2.4 Manual Grounding Evaluation

The accepted architecture is realistic for a 3-hour examination: it needs no installation, no build step and no accounts, and every component is a plain file that can be opened and explained in a five-minute oral defence. The one

real risk — keeping a JavaScript copy of the schema alongside the SQL — was removed by having the browser fetch and execute `database/schema.sql` and `database/seed.sql` directly, so the SQL the examiners read is the SQL the site runs. The cost is a known limitation: the database lives in each visitor's browser, so it is a prototype, not a system of record, and the README says so.

2.5 Final Decision / Refinement

Prompt requested	A lightweight architecture suitable for a 3-hour prototype.
AI output	A browser-based static architecture: HTML/CSS/vanilla JS with SQLite executed in the browser through sql.js.
Team refinement	Retained it; kept C# as a standalone, honestly-labelled exercise instead of pretending it backs the site; made the SQL files the single source of truth.

3. Task 2 — AI-Assisted Frontend Development

Owner: Justin Basilides

3.1 Objective

A responsive, WCAG-conscious event catalog, event detail page and registration form that runs as a static site.

3.2 Prompt Used

Reconstructed task-specific prompt (see the note at the top of this document); it carries the frontend requirements of the Task 1 prompt and excludes database and C# requirements.

ROLE

You are a senior front-end engineer who specialises in accessible, framework-free web interfaces.

CONTEXT

DLSUD EventHub is a campus event registration prototype for De La Salle University-Dasmarias, built in a 3-hour examination and hosted on GitHub Pages. Students must be able to find an event and register for it.

TASK

Build the front end: an event catalog with search and filters, an event detail page, and a registration form (full name, student number, DLSU-D student e-mail, department). Show remaining seats, and show validation feedback for every field.

CONSTRAINTS

- Semantic HTML5 landmarks and heading structure.
- Responsive layout from 320 px upward.
- WCAG 2.1 AA: every form control has a programmatically associated label; ARIA only where HTML has no equivalent; accessible colour contrast; alt text for any informative image; visible validation feedback linked to its field; everything usable by keyboard alone.
- Vanilla HTML, CSS and JavaScript. Relative paths only (GitHub Pages).
- NEGATIVE CONSTRAINT: no framework, no CSS framework, no bundler, no CDN.
- NEGATIVE CONSTRAINT: do not rely on colour alone to convey seat status or errors.

3.3 AI-Generated Output

The generated front end is `frontend/index.html` (catalog), `frontend/event.html` (detail + registration), `frontend/admin.html`, `frontend/css/styles.css` and `frontend/js/{app,events,database,validation,registration}.js`, with a root `index.html` that forwards to `frontend/`. How each requirement is met:

Requirement	Where it is satisfied
Semantic HTML5	<code>header</code> , <code>nav</code> , <code>main</code> , <code>section</code> , <code>article</code> , <code>aside</code> , <code>footer</code> , <code>table / th[scope]</code> , <code>dl</code> , <code>time</code> ; one <code>h1</code> per page, no skipped levels
Accessible forms	every control has a real <code><label for></code> ; hints and errors wired with <code>aria-describedby</code> ; <code>aria-invalid</code> on failure; <code>novalidate</code> so the app owns the messages
Accessible labels	<code>frontend/js/app.js</code> → <code>buildForm()</code> ; filter labels in <code>frontend/index.html</code>
Colour contrast	all text ≥ 4.5:1; gold is used as a border/background accent, with <code>--gold-600</code> reserved for text on white (5.4:1)
Alt text / decoration	informative text is real text; the header logo is an <code></code> beside the wordmark, and the seat bar and alert glyphs are <code>aria-hidden="true"</code>
Event catalog	<code>frontend/index.html</code> + <code>frontend/js/events.js</code> → <code>listEvents()</code> , <code>renderEventCard()</code>
Event registration form	<code>frontend/event.html</code> + <code>frontend/js/app.js</code> → <code>buildForm()</code> , <code>wireForm()</code> , <code>submitRegistration()</code>

Requirement	Where it is satisfied
Responsive UI	fluid grids, <code>clamp()</code> type, breakpoints at 960 px and 620 px; verified at 320/375/768 px with no horizontal overflow
Works as a static site	relative paths only; no API calls; runs from <code>python3 -m http.server</code> or GitHub Pages

Behaviour designed into the flow (all exercised by `tests/browser-checks.mjs`):

The user does this	What happens
Submits an empty form	all four fields report their own message, <code>aria-invalid</code> is set, a summary alert names how many fields need attention, and focus moves to the first invalid field
Enters another school's e-mail	"Only @dlsud.edu.ph student e-mail addresses can register."
Enters a look-alike domain (<code>@dlsud.edu.ph.fake.com</code>)	rejected — the domain is compared in full, never with a suffix match
Pastes 300 characters into a field	<code>maxLength</code> clamps it, and the rules reject anything still too long
Clicks Register repeatedly / very fast	a submitting flag plus a disabled, busy button means one registration; the panel also ignores pointer input for 500 ms after it swaps contents, so a stray second click cannot navigate away from the confirmation
Registers for a full event	the form is not rendered at all; a warning explains it and offers the catalog
Tries to register twice	refused against the e-mail field and in the summary; impossible to commit (<code>UNIQUE (user_id, event_id)</code>)
Reloads mid-flow	the <code>localStorage</code> snapshot restores the database, so a confirmed seat is still confirmed
Opens <code>event.html</code> with no id, <code>id=abc</code> , <code>id=0</code> , <code>id=-5</code> , <code>id=1.5</code> or an unknown id	a "not found" page with an explanation and a link back — never a crash or a blank page
Searches for something with no matches	an empty state that says what to try next, announced in a live region
Opens an event nobody has registered for	all seats shown as available; the admin attendee list shows "No one has registered for this event yet", not a blank table
Passes a bogus <code>?category=</code> or <code>?when=</code> to the catalog	the value is checked against what the <code><select></code> actually offers and ignored if unknown, so the full catalog is shown rather than an empty one
Searches with <code>%</code> , <code>_</code> or <code>'</code> ; <code>DROP TABLE events; --</code>	treated as literal text: <code>LIKE</code> wildcards are escaped and the term is a bound parameter
Clears the filters	everything resets and focus returns to the search box
Uses the browser back button	filters are mirrored into the URL with <code>replaceState</code> , so the catalog view is restored

The user does this	What happens
Presses Enter in the search field	the form submits to the filter handler; the page does not reload
Navigates by keyboard only	skip link first, visible focus ring everywhere, scrollable tables are focusable <code><section></code> s
Resizes to 320 px	single-column layout, full-width buttons, no clipped controls, no horizontal page scroll
Has <code>localStorage</code> blocked or full	the prototype keeps working in memory and says so in a warning — it does not fail the action
Loses <code>database/schema.sql</code> (404)	a readable "could not start" message explaining that the site must be served over http, not opened from the file system

3.4 Manual Corrections / Refinements

Defects found by auditing the generated front end (commit `9f4c5fc`):

Generated behaviour	Correction
The "event not found" page rendered its message as an <code><h3></code> , leaving the page with no <code>h1</code> (flagged by axe-core as <code>page-has-heading-one</code>).	The state renderer takes a heading-level parameter; <code>h1</code> is passed where the state is the page's main content.
Scrollable tables used <code><div role="region" tabindex="0"></code> (flagged by html-validate <code>prefer-native-element</code>).	Replaced with <code><section aria-labelledby tabindex="0"></code> , which has the same role natively.
The filter form had no submit button because filtering runs on <code>input</code> (flagged by html-validate <code>wcag/h32</code>).	Added a real <code>type="submit"</code> Search button; live filtering and the Enter-key handler were kept.
A fast double-click on "Reserve my seat" landed the second click on the "Browse more events" link that replaced the button, so the student left their own confirmation.	Added a 500 ms pointer shield (<code>.panel--settling</code>) when a panel swaps contents; regression check added.
A CSS custom property was written <code>--line-200: #e8ece a;</code> (stray space), producing no colour.	Value corrected; the fallback that had masked it was removed.
Dead code and unused <code>catch (error)</code> bindings (ESLint <code>no-unused-vars</code>).	Removed; ignored catches use optional catch binding.

3.5 Accessibility Verification

Verified in the repository:

- **Labels:** every field is built in `buildForm()` (`frontend/js/app.js`) with a `<label>` whose `htmlFor` matches the control id; hints and error containers are linked through `aria-describedby`; failures set `aria-invalid`; the form uses `novalidate` so the app owns the messages; inputs carry `autocomplete`, `maxlength`, and `inputmode` where relevant. Filter controls in `frontend/index.html` use `<label for>` too.

- **ARIA that exists:** `aria-label` on both `nav` elements and the registration `aside`, `aria-labelledby` on sections, `aria-live / role="status"` for results and field errors, `role="alert"` for the error summary, `aria-busy` on loading regions, `aria-current` for the current page, `aria-hidden` on decorative elements.
 - **Keyboard:** skip link first, `:focus-visible` ring on all interactive elements (`styles.css`), focus moved to the first invalid field and to the confirmation panel.
 - **Contrast / motion / touch:** gold text uses `--gold-600` (5.4:1 on white); `prefers-reduced-motion` is honoured; controls have a 44 px minimum height.
 - **Images:** the only image is the header logo, an `<img alt="">` placed next to the wordmark that already names the site, so it is deliberately treated as decorative; no other informative image exists. The seat bar and alert glyphs are `aria-hidden`.
 - **Tooling run during the build:** `html-validate` (clean), `axe-core 4.10` on six page states at 1280 px and 320 px (0 violations after the `h1` fix), and 91 Playwright checks including keyboard focus order.
 - **Not done:** a manual screen-reader pass (NVDA/VoiceOver). See §6.5.
-

4. Task 3 — Database Design & ERD Generation

Owner: Aian Cuento

4.1 Objective

A relational schema in at least 3NF, executable in SQLite, with an ERD that matches it exactly.

4.2 Prompt Used

Reconstructed task-specific prompt (see the note at the top of this document).

ROLE

You are a database engineer experienced with SQLite and relational modelling.

CONTEXT

DLSUD EventHub lets DLSU-D students register for campus events. Students belong to a department, events have a capacity, and a student may attend many events. The same SQL must run in the browser (sql.js) and in a standalone SQLite client.

TASK

Design the database and produce (1) SQLite-compatible DDL and (2) a Mermaid ERD that matches it.

CONSTRAINTS

- At least 3NF, with at least 3 entities; justify the normal form.
- Primary keys on every table; foreign keys with explicit ON UPDATE/ON DELETE.
- CHECK constraints for the DLSU-D e-mail rule, capacity, dates, categories and status; UNIQUE constraints where justified (including one registration per student per event).
- An index on every foreign-key column.
- The ERD must list the same tables, columns and relationships as the SQL.
- NEGATIVE CONSTRAINT: do not store derived values (remaining seats, registered counts).
- NEGATIVE CONSTRAINT: no MySQL/PostgreSQL-only syntax.

4.3 AI-Generated Output

The generated artefacts are `database/schema.sql` (four tables, one view, six indexes) and `database/seed.sql` (9 events, 8 departments, 112 sample students, 115 registrations, deliberately including a full event, a 39/40 event, an empty event, a past event and a cancelled registration). The schema header comment records the 3NF reasoning; the ERD is in §4.6.

4.4 Manual Corrections / Refinements

Generated behaviour	Correction
<code>CHECK (event_date = strftime('%Y-%m-%dT%H:%M', event_date))</code> . For an unparseable value <code>strftime()</code> returns <code>NULL</code> , <code>x = NULL</code> is <code>NULL</code> , and SQLite treats a <code>NULL</code> CHECK as satisfied, so <code>'12/01/2026'</code> was accepted .	Changed <code>=</code> to <code>IS</code> (<code>schema.sql</code> , <code>events.event_date</code>) with an explanatory comment. Caught by a negative-insert script run against the committed SQL.
Foreign-key indexes.	Not a defect in this project. The three FK indexes were already in the first committed <code>schema.sql</code> (commit 373277a), so no "AI omitted FK indexes" correction is claimed.

4.5 Final 3NF Schema

The authoritative source is `database/schema.sql`; it is not duplicated here to avoid a second copy that could drift.

Form	Why it holds
1NF	Every column holds one atomic value. There is no "events attended" list on a student and no comma-separated attendee column on an event; attendance is one row per student per event in <code>registrations</code> .
2NF	The only composite candidate key is <code>registrations (user_id, event_id)</code> , and the remaining attributes (<code>registered_at</code> , <code>status</code>) depend on that whole key — they describe <i>this student's attendance of this event</i> , not the student alone or the event alone.
3NF	No non-key column determines another non-key column. The college name is the case that would break this: storing <code>department</code> as text on <code>users</code> would make the name depend on the department rather than on the user, so it was extracted into <code>departments</code> and referenced by id. Seat counts are the other case: <code>registered_count</code> and <code>remaining_seats</code> are derivable from <code>registrations</code> and <code>capacity</code> , so they are not stored — they come from the <code>event_availability</code> view. Nothing can drift out of sync because nothing is duplicated.

Constraint and index inventory (all verified present in `schema.sql`; running the schema and seed in SQLite creates the six `idx_*` indexes and `PRAGMA foreign_key_check` returns no rows):

Kind	Where
Primary keys	all four tables (<code>INTEGER PRIMARY KEY AUTOINCREMENT</code>)
Foreign keys	<code>users.department_id → departments</code> ; <code>registrations.user_id → users</code> ; <code>registrations.event_id → events</code> ; all with explicit <code>ON UPDATE / ON DELETE</code> behaviour
NOT NULL	every column in every table — verified with <code>PRAGMA table_info</code> , there are no nullable non-key columns
UNIQUE	<code>departments.code</code> , <code>departments.name</code> , <code>users.student_id</code> , <code>users.email</code> , <code>events (title, event_date)</code> , <code>registrations (user_id, event_id)</code>
CHECK	upper-case department codes; numeric 6–12-digit student numbers; name/title/description/location lengths; the full <code>@dlsud.edu.ph</code> e-mail rule; <code>capacity BETWEEN 1 AND 5000</code> ; strict <code>YYYY-MM-DDTHH:MM</code> dates; <code>category</code> in a fixed list; <code>status</code> in (<code>'confirmed'</code> , <code>'cancelled'</code>)
Indexes on FK columns	<code>idx_users_department_id</code> , <code>idx_registrations_user_id</code> , <code>idx_registrations_event_id</code>
Indexes for frequent queries	<code>idx_registrations_event_status</code> (seat counting), <code>idx_events_event_date</code> (catalog order), <code>idx_events_category</code> (catalog filter)

`PRAGMA foreign_keys = ON` is issued per connection in `schema.sql`, in `database.js` (browser) and in `SQLiteRegistrationRepository.OpenConnection()` (C#), because SQLite leaves foreign keys off by default.

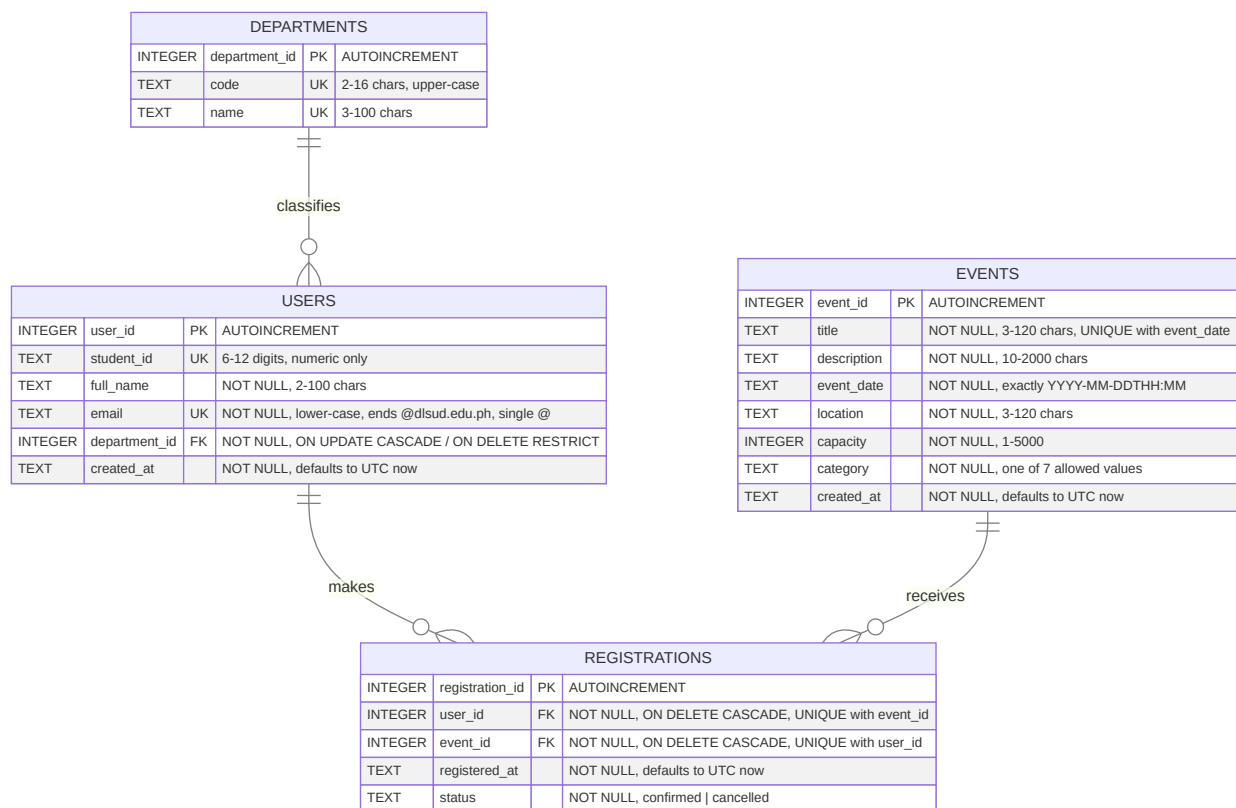
Negative inserts rejected by the constraints (script run against the committed SQL with Python's `sqlite3`):

`@gmail.com` · `@dlsu.edu.ph` · `@dlsud.edu` · `@dlsud.edu.ph.fake.com` ·
`a@b@dlsud.edu.ph` · `@dlsud.edu.ph` (empty local part) · upper-case e-mail ·
non-numeric student number · unknown `department_id` · duplicate

(user_id, event_id) · capacity = -5 · capacity = 0 ·
 event_date = '12/01/2026' · event_date = '2026-13-01T09:00' ·
 category = 'Party' · status = 'maybe' · registration for a non-existent user.

4.6 Mermaid ERD

Written from the final `database/schema.sql` and checked table-by-table, column-by-column and key-by-key against it (tables, column names, PK/FK/UK markers, `ON DELETE` behaviour, and cardinality: one department to many users; one user to many registrations; one event to many registrations).



Plus one derived object, which stores nothing:

EVENT_AVAILABILITY		
INTEGER	event_id	from events
INTEGER	capacity	from events
INTEGER	confirmed_count	COUNT of confirmed registrations
INTEGER	remaining_seats	MAX(capacity - confirmed_count, 0)
INTEGER	is_full	1 when confirmed_count >= capacity

5. Task 4 — Shift-Left Testing, Security & Refactoring

Owner: Calvin Bellen

5.1 Objective

Unit-test the registration business rules, then review and refactor a deliberately flawed C# data-access method for security and resource safety.

5.2 Prompt Used

Reconstructed task-specific prompt (see the note at the top of this document).

ROLE

You are a QA and application-security engineer reviewing C# data-access code.

CONTEXT

DLSUD EventHub registers students for events. The following legacy method is deliberately flawed:

```
public bool RegisterStudent(string email, string eventId)
{
    SqlConnection conn = new SqlConnection("Server=.;Database=EventHub;User
Id=sa;Password=P@ssw0rd123;");
    conn.Open();
    string sql = "INSERT INTO Registrations (Email, EventId) VALUES ('" + email +
    "', " + eventId + ")";
    SqlCommand cmd = new SqlCommand(sql, conn);
    int rows = cmd.ExecuteNonQuery();
    string name = cmd.ExecuteScalar().ToString();
    return rows > 0;
}
```

TASK

- (1) Write xUnit tests for: DLSU-D e-mail validation, an invalid e-mail, available seats, a full event, and duplicate registration.
- (2) Review the method for SQL injection, string-concatenated SQL, resource disposal, null handling and hard-coded credentials, and explain each finding.
- (3) Refactor it.

CONSTRAINTS

- Parameterised queries with safe parameter binding.
- using / using var for every connection, command, reader and transaction.
- Safe handling of null and DBNull results.
- No real credentials; the connection string comes from configuration.
- Tests assert requirements (behaviour), not implementation details.
- NEGATIVE CONSTRAINT: do not concatenate user input into SQL anywhere.
- NEGATIVE CONSTRAINT: do not weaken or skip a test to make it pass.

5.3 AI-Generated Output

The output is `backend/RegistrationService.cs` (business rules, a mockable `IRegistrationRepository`, `RegistrationService`, and `SQLiteRegistrationRepository`) and `tests/RegistrationServiceTests.cs`. The original flawed method is preserved in a comment at the top of `RegistrationService.cs` as the "before".

5.4 Security Findings

Findings against the original method, each present in the preserved code:

#	Finding	Evidence in the original
1	SQL injection / string-concatenated SQL	<code>"... VALUES ('" + email + "', " + eventId + ")"</code>
2	Hard-coded credentials	<code>User Id=sa;Password=P@ssw0rd123;</code> in source
3	Connection not disposed	<code>new SqlConnection(...)</code> , never closed; leaks on exception
4	Command not disposed	<code>new SqlCommand(...)</code> , never disposed
5	Null dereference	<code>cmd.ExecuteScalar().ToString()</code> throws when the result is null
6	No business rules	no capacity, duplicate or e-mail check

5.5 Refactored Solution

`backend/RegistrationService.cs`:

Finding	Fix in the final file
1	Every statement uses bound parameters (<code>\$eventId</code> , <code>\$email</code> , ...) through <code>command.Parameters.AddWithValue</code> . Each <code>CommandText</code> is a constant literal.
2	<code>SQLiteRegistrationRepository(string connectionString)</code> receives the connection string from the caller and throws if it is null/blank. The file contains no credentials.
3–4	<code>using SQLiteConnection</code> , <code>using SQLiteCommand</code> , <code>using SQLiteDataReader</code> and <code>using SQLiteTransaction</code> in every method.
5	<code>ExecuteScalar()</code> results are checked for <code>null</code> and <code>DBNull.Value</code> ; <code>reader.Read()</code> returning false returns <code>null</code> ; <code>reader.IsDBNull(i)</code> guards column reads; an unparseable stored date is handled explicitly.
6	<code>RegistrationRules.Decide</code> runs before any write; the insert is <code>INSERT ... SELECT ... WHERE (SELECT COUNT(*) ...) < (SELECT capacity ...)</code> , so capacity is enforced by the database, and <code>UNIQUE (user_id, event_id)</code> blocks duplicates. Writes run in a transaction with an explicit <code>Rollback()</code> on the "nothing inserted" path.

5.6 Unit Test Verification

`tests/RegistrationServiceTests.cs` — xUnit + Moq, the repository mocked with `MockBehavior.Strict` so that any unexpected database call fails the test (this is how "invalid input never reaches the database" is asserted).

Counted from the file: **22 [Fact] tests and 6 [Theory] tests with 44 [InlineData] cases.**

Requirement under test	Tests
Valid DLSU-D e-mail accepted	6 cases, including a padded mixed-case address
External / look-alike / malformed e-mail rejected	22 cases, including <code>@dlsu.edu.ph</code> , <code>@dlsud.edu</code> , <code>@dlsud.edu.ph.fake.com</code> , <code>a@b@...</code> , consecutive dots, an injection payload, and <code>null</code>
Seats available	<code>99/100</code> allowed; <code>39/40</code> allowed and reported as one seat left
Event at capacity	<code>100/100</code> refused, 0 seats remaining, nothing written
Corrupted state	<code>140/100</code> refused and reported as 0 seats, never negative; <code>capacity <= 0</code> is an invalid event; a negative count is an invalid event
Duplicate registration	refused, nothing written; and reported as <code>Duplicate</code> rather than <code>EventFull</code> when both are true
Race on the last seat	the conditional insert returns <code>null</code> → reported as full, no receipt issued
Identity conflicts	e-mail on file under a different student number, and student number on file under a different e-mail, are both refused without writing
Invalid student details	7 cases (empty, too short, injection payload, bad student numbers) refused with no database call at all
Unknown / invalid event	unknown id, <code>0</code> and <code>-1</code> refused; <code>0</code> and <code>-1</code> without touching the database
Past event	refused, nothing written
Normalisation	a padded, mixed-case e-mail, a hyphenated student number and a double-spaced name reach the repository in canonical form
Guard clause	a null repository throws <code>ArgumentNullException</code> at construction

`tests/validation.test.mjs` pins the same e-mail, seat and duplicate rules against the JavaScript the website runs.

Execution status — be precise.

`node --test tests/validation.test.mjs` was run and gives **31 tests, 31 passed** (re-run while preparing this document).

`dotnet test tests/DlsudEventHub.Tests.csproj` has **not** been executed: the build environment has no .NET SDK. The C# tests were reviewed by hand and

are **not** claimed to have passed. Run the command on a machine with the .NET 8 SDK and record the result before submitting.

5.7 Manual Corrections / Refinements

Generated behaviour	Correction
<code>FindStudentByEmail</code> / <code>FindStudentByStudentId</code> shared a helper that built <code>CommandText</code> as <code>"SELECT ... WHERE " + predicate + " LIMIT 1;"</code> . The predicate was a class-chosen constant, not user input, so it was not exploitable, but <code>CommandText</code> was no longer a literal and "no SQL is concatenated" was not literally true.	Replaced with two complete constant statements, each with its own bound parameter (commit <code>9f4c5fc</code>).
The JavaScript name rule used the Latin-1 range <code>[A-Za-zÀ-ÿÑñ. '\-]</code> while the C# rule used <code>[\p{L}. '\-]</code> , so one name could pass one layer and fail the other.	JavaScript aligned to <code>/^[\p{L}. '\-]+\$/u</code> ; a mirroring note added to both files.
Resource disposal.	Not a defect in this project. <code>using</code> declarations were already in the first committed <code>RegistrationService.cs</code> (commit <code>373277a</code>), so no "AI did not dispose resources" correction is claimed.

6. Task 5 — Group Integration & Verification Report

Owner: Timothy Delmoro, with contributions from all members.

6.1 Integration Summary

Deliverable	Location	Owner
Requirements, architecture, RCTC prompt	§2 of this file; <code>README.md</code> (Architecture)	Timothy Delmoro
Front end (catalog, detail, registration, admin)	<code>frontend/</code> , <code>root index.html</code>	Justin Basilides
Schema, seed data, ERD	<code>database/schema.sql</code> , <code>database/seed.sql</code> , §4.6	Aian Cuento
C# service, unit tests, security review	<code>backend/RegistrationService.cs</code> , <code>tests/</code>	Calvin Bellen
Integration, setup, disclosure, verification log	this section, <code>README.md</code>	Timothy Delmoro

The pieces connect through one contract: the browser executes `database/schema.sql` and `database/seed.sql` through `sql.js`, and the C# service

and tests implement the same rules against the same schema. The site does not call the C# code.

6.2 Setup Instructions

1. **Get the code.** `git clone https://github.com/eximno/dlsud-eventhub.git` then `cd dlsud-eventhub` (or use *Code* → *Download ZIP* on GitHub and unzip).
2. **Run the site locally.** It must be served over http, not opened as a file: `python3 -m http.server 8000`, then open <http://localhost:8000/>. Serve the repository **root**, not `frontend/`. Nothing needs installing or building.
3. **Public site.** After enabling Pages (Settings → Pages → *Deploy from a branch* → `main` → `/ (root)`), the site is at <https://eximno.github.io/dlsud-eventhub/>. Not verified on the real Pages URL — see §6.5.
4. **Inspect the SQLite database.** With the `sqlite3` CLI:
`sqlite3 /tmp/eventhub.db < database/schema.sql`,
`sqlite3 /tmp/eventhub.db < database/seed.sql`, then
`sqlite3 /tmp/eventhub.db "SELECT title, capacity, confirmed_count, remaining_seats FROM event_availability;"`.
Without it, use the Python snippet in `README.md` ("Inspecting the SQL outside the browser"). In the running site, the Admin page shows the same data.
5. **Run the tests.** JavaScript (needs Node 18+):
`node --test tests/validation.test.mjs`. C# (needs the .NET 8 SDK):
`dotnet test tests/DlsudEventHub.Tests.csproj`.
6. **Where each deliverable is:** see the table in §6.1.

6.3 AI Disclosure Statement

An AI assistant (Claude) was used for: requirements and architecture ideation (Task 1); front-end generation and refinement (Task 2); database schema and seed generation and the Mermaid ERD (Task 3); unit-test drafting, security analysis and the C# refactor (Task 4); and drafting this documentation and the README.

The team reviewed, executed where possible, and corrected the output; the corrections that can be traced to the repository are in the Verification Log. This does **not** mean every result was changed — some generated artefacts were kept as produced after review (for example the foreign-key indexes and the `using` declarations, which needed no correction). Manual verification claims in this document are limited to what §6.5 lists as executed. The Task 2–4 prompts are reconstructions, as stated at the top of this document.

6.4 Verification Log

Instances where AI-generated output was wrong, incomplete or over-built, and what was done about it. *How it was caught* names the tool or check. Entries 1–6 and 9–10 are traceable to commit `9f4c5fc` ; entries 11–13 come from later audit commits on `main` ; entries 7 and 8 are team-recorded design-review findings that predate the first commit and cannot be reproduced from git history. *Member responsible* follows the roster's task ownership.

#	Task	AI flaw / limitation	Manual correction	How it was caught	Member responsible
1	3	The generated schema wrote <code>CHECK (event_date = strftime('%Y-%m-%dT%H:%M', event_date))</code> . For an unparseable value <code>strftime()</code> returns <code>NULL</code> , <code>'x' = NULL</code> evaluates to <code>NULL</code> , and SQLite treats a <code>NULL CHECK</code> as satisfied — so <code>'12/01/2026'</code> was accepted into the database.	Changed <code>=</code> to the <code>IS</code> operator, which compares against <code>NULL</code> correctly, and added a comment explaining why. <code>'12/01/2026'</code> , <code>'2026-12-01'</code> , <code>'2026-13-01T09:00'</code> , a value with seconds, and <code>' '</code> are now all rejected.	A negative-constraint test script run against the committed SQL: 15 of 16 bad inserts were rejected and this one was not.	Aian Cuento
2	2	The generated registration flow replaced the form with a confirmation panel the instant the insert succeeded. A fast double-click therefore landed its second click on the "Browse more events" link that had appeared where the submit button was, and the student was navigated to the catalog away from their own reference number . The data was correct — exactly one registration — but the user lost the confirmation.	Added a 500 ms pointer shield (<code>.panel - - settling</code>) applied whenever the panel swaps its contents, so links and buttons inside it ignore pointer input briefly; keyboard users are unaffected because focus is moved to the panel. Added a regression check asserting that rapid repeat clicks neither duplicate the registration nor change the URL.	An end-to-end check that triple-clicks the submit button and then asserts the registration count; it aborted because the page had navigated.	Justin Basilides
3	2	The generated "event not found" page rendered its message with an <code><h3></code> inside the main article, leaving that page with no level-one heading — a WCAG heading-structure failure on the one page a mistyped link lands on.	Gave the state renderer an explicit heading-level parameter and passed <code>h1</code> where the state is the page's main content (and <code>h2</code> for the admin failure state, which sits under the page <code>h1</code>).	<code>axe-core</code> 4.10 run over every page and every state: 1 violation, <code>page-has-heading-one</code> .	Justin Basilides

#	Task	AI flaw / limitation	Manual correction	How it was caught	Member responsible
4	2	The generated admin markup used <code><div role="region" aria-labelledby="..." tabindex="0"></code> for the horizontally scrollable tables — ARIA standing in for an element HTML already has.	Replaced both with <code><section aria-labelledby="..." tabindex="0"></code> , which carries the same role natively.	<code>html-validate</code> (<code>prefer-native-element</code>).	Justin Basilides
5	2	The generated filter form had no submit button at all, because filtering happens on <code>input</code> . That fails WCAG technique H32 and leaves anyone who expects to press a button with nothing to press.	Added a real <code>type="submit"</code> Search button next to Clear filters , kept the live filtering, and kept the <code>submit</code> handler that already prevented a page reload on Enter.	<code>html-validate</code> (<code>wcag/h32</code>).	Justin Basilides
6	4	The generated JavaScript name rule used the Latin-1 range <code>[A-Za-zÀ-ÿÑñ. '\-]</code> while the generated C# rule used <code>[\p{L}. '\-]</code> . The same name could pass one layer and fail the other , which is exactly the kind of silent divergence the three-layer design is supposed to avoid.	Aligned the JavaScript to <code>/^[\\p{L}. '\\-]+\$/u</code> so both implementations apply one rule, and noted the mirroring requirement in both files.	Line-by-line comparison of the JavaScript rules against the C# rules while writing the second test suite.	Calvin Bellen
7	4	The first C# repository design checked capacity in application code and then inserted unconditionally. Two concurrent requests for the last seat would both pass the check and both insert, over-subscribing the event.	The insert now re-counts confirmed seats in its own <code>WHERE</code> clause, so the database enforces capacity; zero rows inserted is reported as "the last seat was taken while you were registering". A test drives that null-return path.	Design review of the write path: asking "what happens if two of these run at once?"	Calvin Bellen
8	1 / 2	The first architecture proposal was a React SPA with an Express/ASP.NET REST API, PostgreSQL, JWT auth, Docker Compose and a CI pipeline — none of which can run on GitHub Pages, and none of which is finishable in a 3-hour laboratory slot.	Rejected it, tightened the prompt with explicit negative constraints, and implemented static HTML/CSS/JS + SQLite via <code>sql.js</code> . The repository has zero runtime dependencies beyond the two vendored <code>sql.js</code> files.	Checking the proposal against the hard requirement that the site be reachable on GitHub Pages.	Timothy Delmoro

#	Task	AI flaw / limitation	Manual correction	How it was caught	Member responsible
9	2	A generated CSS custom property was written as <code>--line-200: #e8ece a;</code> — an invalid value with a stray space, which silently produced no colour wherever the token was used.	Corrected the value and removed the defensive fallback that had been masking it.	Reading the generated stylesheet rather than trusting it, after a brace-balance check passed.	Justin Basilides
10	2	Generated JavaScript left dead code behind: an unused object built in the blur handler, and five <code>catch (error)</code> blocks whose binding was never read.	Removed the dead object and converted the deliberately-ignored catches to ES2019 optional catch binding, so the intent ("this failure is expected and ignorable") is visible rather than implied.	ESLint 9 (<code>no-unused-vars</code>).	Justin Basilides
11	2	The event detail page's "date / venue / category / capacity" block was a <code><dl></code> laid out as a CSS grid with bare <code><dt></code> and <code><dd></code> children. Because each element is its own grid item, labels and values landed in different columns . The markup was valid and the automated checks were green; it was simply wrong on screen.	Wrapped each <code>dt / dd</code> pair in a <code><div></code> (valid inside a <code><dl></code>) so the pair is one grid item, and said why in a comment.	Rendering the page in a real browser and looking at it ; no linter, test or accessibility checker reported it.	Justin Basilides
12	2	Every field error message sat inside its own <code>aria-live="polite"</code> region <i>and</i> was summarised by a <code>role="alert"</code> box, so a screen reader announced each error twice on submit.	Removed the live regions from the field messages; they remain reachable through <code>aria-describedby</code> when focus lands on the field, and the summary alert announces the count once.	Reading the generated ARIA against what it would actually do.	Justin Basilides

#	Task	AI flaw / limitation	Manual correction	How it was caught	Member responsible
13	2 / 4	In the browser path, capacity was enforced only by the JavaScript decision taken immediately before the <code>INSERT</code> , so "the database enforces capacity", true of the C# deliverable, was not true of the site.	Gave the browser the same capacity-guarded <code>INSERT ... SELECT ... WHERE (count) < (capacity)</code> and made a zero-row insert roll the transaction back. A browser check disables the JavaScript rule, submits against a full event, and asserts the event stays at 40/40 with no orphan student record.	Comparing the two implementations' write paths while documenting them.	Calvin Bellen

Not entered as flaws (checked against the repository and found not to have happened): missing `aria-label` /labels on required inputs (labels were in the first commit), omitted foreign-key indexes (present in the first commit), and undisposed database resources (`using` was present in the first commit).

6.5 Final Verification

Executed, with results

Check	Result
<code>node --test tests/validation.test.mjs</code>	31 tests, 31 passed (re-run for this document)
<code>schema.sql + seed.sql</code> loaded in SQLite; <code>PRAGMA foreign_key_check</code>	no violations; 9 events, 112 users, 115 registrations; all six <code>idx_*</code> indexes exist
html-validate, ESLint 9, axe-core 4.10, <code>node tests/browser-checks.mjs</code> (Chromium)	Reported clean / 0 violations / 95 of 95 checks by the build session (<code>README.md</code> , commit messages); not re-run for this document

Not executed

Check	Status
<code>dotnet test</code>	Not run — no .NET SDK in the build environment. C# test results are unknown.
Screen-reader pass (NVDA / VoiceOver)	Not done.
Live GitHub Pages URL	Not verified — Pages must be enabled by the repository owner. The <i>path layout</i> Pages produces has been exercised, though: the browser suite was run twice, once at a server root and once under a <code>/dlsud-eventhub/</code> subpath, passing 95 of 95 both times.

Known limitations: browser-side SQLite is local to each visitor and not a secure system of record; the admin page has no authentication. Both are stated in `README.md` and on the admin page.

Security audit (supporting)

Risk	Status in this prototype
SQL injection	Mitigated. All SQL in <code>frontend/js/*.js</code> and <code>backend/RegistrationService.cs</code> uses bound parameters. Verified by grep and by driving injection payloads through the search box, the form fields and the <code>?id=</code> parameter.
LIKE wildcard abuse	Mitigated. <code>%</code> and <code>_</code> in search text are escaped and the query uses <code>ESCAPE '\'</code> , so they match literally instead of matching everything.
XSS / unsafe DOM insertion	Mitigated. <code>innerHTML</code> , <code>outerHTML</code> , <code>insertAdjacentHTML</code> , <code>document.write</code> , <code>eval</code> and <code>new Function</code> do not appear anywhere in the repository. Every value is written with <code>createElement + textContent</code> , including the search-term highlighter, which splits text and builds <code><mark></code> nodes.
Unsafe URLs	Mitigated. Event links are built with <code>encodeURIComponent</code> ; <code>javascript:</code> and <code>data:</code> URLs are never constructed from input.
Invalid identifiers	Mitigated. <code>?id=</code> is accepted only as a plain positive integer (<code>/^[0-9]{1,9}\$/</code>), so <code>1.5</code> , <code>1e3</code> , <code>0x2</code> , <code>-5</code> and <code>1</code> ; <code>DROP TABLE events</code> all produce a "not found" page.
Oversized input	Mitigated. <code>maxLength</code> in the UI, explicit ceilings in the rules (names 100, e-mail 100, student number 12, search 80) and <code>CHECK</code> length constraints in the schema.
Duplicate submission	Mitigated. Submitting flag, disabled busy button, re-check inside the transaction, <code>UNIQUE (user_id, event_id)</code> , and a pointer shield after the panel swaps so a stray repeat click cannot act on the new content.
Over-subscription by a tampered client	Mitigated. Both the browser and the C# service insert through <code>INSERT ... SELECT ... WHERE (confirmed count) < (capacity)</code> , so the database refuses the row even when the client-side rule is removed. A browser check does exactly that and asserts the event stays at capacity with no orphan record.
Manipulated registration state	Partly mitigated, and documented. A corrupted or hand-edited database cannot produce negative seats or re-open a full event (the view clamps at 0, the rules treat over-subscription as full, and a non-positive capacity makes the event invalid). It cannot be <i>prevented</i> , because the data lives in the visitor's own browser.
Prototype pollution	Not applicable in practice. No deep-merge, no <code>Object.assign</code> over user input, no dynamic property assignment from parsed input; query parameters are read individually and validated.
Credential exposure	None. No passwords, keys or tokens anywhere in the repository. The C# connection string is injected from configuration.
Insecure local data assumptions	Documented, not hidden. The README and the admin page both state that browser-side SQLite is not authoritative and not secure.
Authentication / authorisation	Absent by design, and labelled. The admin page carries a banner saying it has no authentication and provides no real access control.

The prototype has no server and therefore no trust boundary; the injection and XSS mitigations are habits that matter once a server exists, which is what `backend/RegistrationService.cs` demonstrates.